The `span_near` query accepts multiple clauses; we also have `span_term` queries trying to match our terms. Additionally, because we know these two words are three positions apart, we can provide this difference in the form of the `slop` attribute. The `slop` attribute permits the acceptable maximum positional difference between the words. For example, the query specifies that the two words can be separated by a maximum of three positions. We can increase the slop (say, `"slop": 10`) to reduce the strict constraints so the query has a greater chance of success. However, `span` queries are good at finding exact words at exact positions, so increase the `slop` attribute judiciously.

In addition to using the `slop` attribute, we can define the order of the words. Continuing with the same example, if order is unimportant, we can return a positive result even if we request `span_near` between *Aristotle* and *Plato* (instead of *Plato* and *Aristotle*). If order is important, we can set a Boolean for the `in_order` flag. The `in_order` attribute can be set to *true* or *false*; when it's set to *true*, as in listing 12.16, the order in which the words are indexed is taken into account.

12.8.4 The span_within query

The next use case for `span` queries is when we want to find a word between two words. For example, suppose we want to find documents where *Aristotle* is between two words: *friend* and *friend*, as shown in figure 12.11.

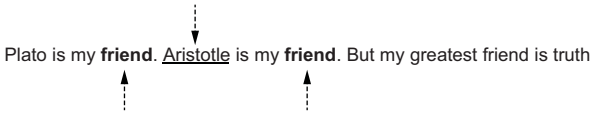


Figure 12.11 Finding a word if it exists between other words

We can use the `span_within` query type for this purpose. Let's look at the query in the following listing and then dissect it.

Listing 12.17 Searching for a word between other words

```
GET quotes/_search
{
  "query": {
    "span_within": {
      "little": {
        "span_term": {
          "quote": "aristotle"
        }
      },
      "big": {
        "span_near": {
          "clauses": [
            {
```

A span_within query consisting of two blocks: little and big

Defines the search word

Encloses the little block